Retrieval-Augmented Generation: A Technical Overview

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that enhances large language model responses by grounding them in external knowledge retrieved at inference time. Rather than relying solely on knowledge encoded in model parameters, RAG systems dynamically fetch relevant documents and use them as context for generation.

The key insight behind RAG is that language models have a fixed knowledge cutoff and finite parameter capacity. By coupling them with a retrieval system, we can provide up-to-date, domain-specific information without expensive model retraining or fine-tuning.

Core Components of a RAG Pipeline

Document ingestion is the first stage, where raw documents (PDFs, text files, web pages) are loaded, cleaned, and prepared for indexing. The quality of document preprocessing significantly impacts downstream retrieval performance.

Chunking divides documents into smaller, semantically meaningful segments. The chunk size and overlap must be tuned carefully — chunks that are too small lose context, while chunks that are too large dilute relevance signals.

Embedding converts text chunks into dense vector representations that capture semantic meaning. Models like sentence-transformers encode similar meanings into nearby regions of the vector space, enabling semantic similarity search.

Vector storage indexes these embeddings for efficient similarity search. Popular options include FAISS for local in-memory search, Pinecone and Weaviate for cloud-scale deployments, and ChromaDB for persistent local storage.

Retrieval Strategies

Dense retrieval uses learned embedding models to find semantically similar chunks. When a user asks a question, the query is embedded into the same vector space, and the nearest chunks are retrieved by cosine or inner product similarity.

Sparse retrieval methods like BM25 use keyword frequency statistics to score document relevance. BM25 excels at exact keyword matching and is extremely fast, but struggles with paraphrases and semantic variations.

Hybrid retrieval combines dense and sparse signals, typically via Reciprocal Rank Fusion (RRF). This approach often outperforms either method alone, especially on diverse query types.

Evaluation Metrics

Hit Rate at k measures whether the relevant document appears anywhere in the top-k retrieved results. This is a coarse but important metric — if the retriever never finds the right chunk, the generator cannot produce a correct answer.

Mean Reciprocal Rank (MRR) rewards retrievers that rank the correct document higher. If the relevant chunk is ranked first, MRR = 1.0; if ranked second, MRR = 0.5; and so on. MRR provides a more nuanced view than hit rate alone.

End-to-end evaluation measures the final answer quality using metrics like RAGAS, which scores faithfulness (is the answer grounded in context?) and answer relevance (does it address the question?).

How FAISS Works

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search over dense vectors. It supports both exact and approximate nearest-neighbor search, with trade-offs between speed and accuracy.

IndexFlatIP performs exact inner product search with no approximation. For normalized vectors, inner product equals cosine similarity, making this the recommended index type for semantic search at small-to-medium scale.

For large-scale deployments, FAISS offers IndexIVFFlat and IndexHNSW, which sacrifice a small amount of accuracy for dramatically faster search. These are appropriate when indexing millions of vectors.

Challenges and Limitations

Chunking strategy significantly impacts RAG performance. If the answer spans multiple chunks or falls at a chunk boundary, retrieval may fail even with perfect embeddings. Overlapping chunks partially mitigate this.

Query-document mismatch occurs when the user's phrasing differs significantly from how information is expressed in documents. This is where dense retrieval with good embedding models outperforms keyword-based approaches.

Hallucination remains a risk even in RAG systems. If retrieved chunks are tangentially relevant rather than directly answering the question, the LLM may still generate plausible-sounding but incorrect answers. Strict prompting to stay grounded in context reduces this risk.